Testes de Mesa: Algoritmos de Ordenação

PSC II · Engenharia de Software · Centro Universitário Campo Real
Prof. Giovane Galvão · Aula 19 · Aluno: Gabriel Goes

Como ler este documento

Os cinco algoritmos (Bubble, Selection, Insertion, Merge e Quick Sort) foram implementados em C++ com base no código da aula e usam o mesmo array de entrada: **[5 3 8 1 2]**. Cada teste de mesa mostra o passo a passo e as colunas **Comp.** e **Trocas** trazem o total acumulado até aquela linha.

Como contei as trocas em cada algoritmo:

Bubble, Selection e Quick Sort: cada chamada de swap conta como uma troca. No Quick Sort, isso inclui o swap de um elemento com ele mesmo, que também acontece no código da aula.

Insertion Sort: não troca de verdade, ele desloca elementos uma posição para a direita. Cada deslocamento conta como uma troca.

Merge Sort: não troca posições, ele copia os valores das listas auxiliares L e R de volta para o array. Cada cópia conta como uma troca.

O Quick Sort usa o último elemento como pivô (sem o rand() do slide), para o resultado ser sempre o mesmo.

1. Bubble Sort

A cada passagem ele compara pares vizinhos e troca quando estão fora de ordem, assim o maior elemento vai para o fim. Se uma passagem inteira não fizer nenhuma troca, o array já está ordenado e o laço para.

Entrada: **[5 3 8 1 2]**

Passagem	j	Comparação	Ação	Array depois	Comp.	Trocas
1	0	5 > 3	troca	3 5 8 1 2	1	1
1	1	5 > 8	não troca	3 5 8 1 2	2	1
1	2	8 > 1	troca	3 5 1 8 2	3	2
1	3	8 > 2	troca	3 5 1 2 8	4	3
2	0	3 > 5	não troca	3 5 1 2 8	5	3
2	1	5 > 1	troca	3 1 5 2 8	6	4
2	2	5 > 2	troca	3 1 2 5 8	7	5
3	0	3 > 1	troca	1 3 2 5 8	8	6
3	1	3 > 2	troca	1 2 3 5 8	9	7
4	0	1 > 2	não troca	1 2 3 5 8	10	7

Array final: **[1 2 3 5 8]**
Total: **10 comparações** e **7 trocas**. Na passagem 4 nenhuma troca aconteceu, então o laço parou.

2. Selection Sort

A cada passagem ele procura o menor elemento da parte que ainda não está ordenada e coloca esse elemento no início dela. Faz sempre o mesmo número de comparações e poucas trocas.

Entrada: **[5 3 8 1 2]**. A coluna Mínimo mostra a posição do menor elemento encontrado até o momento.

Passagem	j	Comparação	Mínimo	Ação	Array depois	Comp.	Trocas
1	1	3 < 5	pos 1		5 3 8 1 2	1	0
1	2	8 < 3	pos 1		5 3 8 1 2	2	0
1	3	1 < 3	pos 3		5 3 8 1 2	3	0
1	4	2 < 1	pos 3		5 3 8 1 2	4	0
1			pos 3	troca pos 0 e 3	1 3 8 5 2	4	1
2	2	8 < 3	pos 1		1 3 8 5 2	5	1
2	3	5 < 3	pos 1		1 3 8 5 2	6	1
2	4	2 < 3	pos 4		1 3 8 5 2	7	1
2			pos 4	troca pos 1 e 4	1 2 8 5 3	7	2
3	3	5 < 8	pos 3		1 2 8 5 3	8	2
3	4	3 < 5	pos 4		1 2 8 5 3	9	2
3			pos 4	troca pos 2 e 4	1 2 3 5 8	9	3
4	4	8 < 5	pos 3		1 2 3 5 8	10	3
4			pos 3	sem troca	1 2 3 5 8	10	3

Array final: **[1 2 3 5 8]**
Total: **10 comparações** e **3 trocas**. As comparações seriam as mesmas para qualquer array de 5 elementos.

3. Insertion Sort

Ele pega cada elemento (a chave) e desloca para a direita os elementos maiores que ela, até achar o lugar certo para inserir. É como ordenar cartas na mão.

Entrada: **[5 3 8 1 2]**. Enquanto os elementos são deslocados, o array fica com um valor repetido temporariamente, até a chave ser inserida.

Passagem	Chave	j	Comparação	Ação	Array depois	Comp.	Trocas
1	3	0	5 > 3	desloca	5 5 8 1 2	1	1
1	3			insere 3 na pos 0	3 5 8 1 2	1	1
2	8	1	5 > 8	para	3 5 8 1 2	2	1
2	8			insere 8 na pos 2	3 5 8 1 2	2	1
3	1	2	8 > 1	desloca	3 5 8 8 2	3	2
3	1	1	5 > 1	desloca	3 5 5 8 2	4	3
3	1	0	3 > 1	desloca	3 3 5 8 2	5	4
3	1			insere 1 na pos 0	1 3 5 8 2	5	4
4	2	3	8 > 2	desloca	1 3 5 8 8	6	5
4	2	2	5 > 2	desloca	1 3 5 5 8	7	6
4	2	1	3 > 2	desloca	1 3 3 5 8	8	7
4	2	0	1 > 2	para	1 3 3 5 8	9	7
4	2			insere 2 na pos 1	1 2 3 5 8	9	7

Array final: **[1 2 3 5 8]**
Total: **9 comparações** e **7 trocas** (deslocamentos).

4. Merge Sort

Ele divide o array ao meio até sobrar um elemento só e depois junta as metades de forma ordenada, sempre comparando o início das listas L (esquerda) e R (direita).

Entrada: **[5 3 8 1 2]**

Divisões, na ordem em que acontecem:

• [0..4] divide em [0..2] e [3..4]

• [0..2] divide em [0..1] e [2..2]

• [0..1] divide em [0..0] e [1..1]

• [3..4] divide em [3..3] e [4..4]

Junções (merge):

Etapa	L	R	Comparação	Cópia	Array depois	Comp.	Trocas
1	5	3	5 <= 3	copia 3 de R	3 3 8 1 2	1	1
				sobra em L: copia 5	3 5 8 1 2	1	2
2	3 5	8	3 <= 8	copia 3 de L	3 5 8 1 2	2	3
			5 <= 8	copia 5 de L	3 5 8 1 2	3	4
				sobra em R: copia 8	3 5 8 1 2	3	5
3	1	2	1 <= 2	copia 1 de L	3 5 8 1 2	4	6
				sobra em R: copia 2	3 5 8 1 2	4	7
4	3 5 8	1 2	3 <= 1	copia 1 de R	1 5 8 1 2	5	8
			3 <= 2	copia 2 de R	1 2 8 1 2	6	9
				sobra em L: copia 3	1 2 3 1 2	6	10
				sobra em L: copia 5	1 2 3 5 2	6	11
				sobra em L: copia 8	1 2 3 5 8	6	12

Array final: **[1 2 3 5 8]**
Total: **6 comparações** e **12 trocas** (cópias para o array).

5. Quick Sort

Ele escolhe um pivô (aqui, o último elemento), coloca os menores ou iguais ao pivô à esquerda e o pivô na sua posição final. Depois repete o processo para os dois lados.

Entrada: **[5 3 8 1 2]**. A coluna Partição mostra a faixa de posições que está sendo particionada.

Partição	Pivô	j	Comparação	Ação	Array depois	Comp.	Trocas
[0..4]	2	0	5 <= 2	não troca	5 3 8 1 2	1	0
[0..4]	2	1	3 <= 2	não troca	5 3 8 1 2	2	0
[0..4]	2	2	8 <= 2	não troca	5 3 8 1 2	3	0
[0..4]	2	3	1 <= 2	troca pos 0 e 3	1 3 8 5 2	4	1
[0..4]	2			pivô: troca pos 1 e 4	1 2 8 5 3	4	2
[2..4]	3	2	8 <= 3	não troca	1 2 8 5 3	5	2
[2..4]	3	3	5 <= 3	não troca	1 2 8 5 3	6	2
[2..4]	3			pivô: troca pos 2 e 4	1 2 3 5 8	6	3
[3..4]	8	3	5 <= 8	troca pos 3 e 3	1 2 3 5 8	7	4
[3..4]	8			pivô: troca pos 4 e 4	1 2 3 5 8	7	5

Array final: **[1 2 3 5 8]**
Total: **7 comparações** e **5 trocas**. A partição [0..0] tem um elemento só, então não faz nada.

Resumo

Resultado dos cinco algoritmos para a entrada [5 3 8 1 2]:

Algoritmo	Comparações	Trocas
Bubble Sort	10	7
Selection Sort	10	3
Insertion Sort	9	7
Merge Sort	6	12
Quick Sort	7	5

Os números de trocas não são diretamente comparáveis entre todos, porque no Insertion Sort contam deslocamentos e no Merge Sort contam cópias, como explicado no início.